Camet ERP

Hotel and Restaurant Working Flow Analysis

Prepared from the local codebase on 14 August 2026. Scope: React frontend routes and screens, Express routes, Mongoose models, hotel/restaurant controllers, helper services, reports, and accounting integration.

Executive View

The application runs hotel front-office, restaurant/KOT, and accounting flows as one integrated ERP module. Hotel operations manage room masters, booking, check-in, check-out, room status, advances, guest identity, tariff/tax calculation, receipts, checkout sales, reports, and night-audit locking. Restaurant operations manage item masters, tables, KOT creation, kitchen batches, dine-in/takeaway/room-service sales, split payment, complimentary handling, direct sales, and transfer of food bills to checked-in rooms.

Important architecture note: most hotel and restaurant APIs are mounted from the secondary-user router and protected with authentication. The implementation is transactional in the critical write flows, using Mongoose sessions around room creation, KOT generation, checkout conversion, receipt creation, and settlement updates.

Layer	Main Files	Purpose
Frontend routing	frontend/src/routes/Routers.jsx	Maps protected secondary-user screens for hotel dashboard, booking, check-in/out, KOT, tables, reports, and settings.
Hotel UI	frontend/src/pages/Hotel	Operational screens for room setup, bookings, check-in/out, bill summary, print pages, dashboard, and reports.
Restaurant UI	frontend/src/pages/Restuarant	KOT page, dashboard, item registration/list, table selection/master, print helpers, order cards, reports.
Backend routes	backend/routes/secondaryUserRouters.js	Authenticated API surface for hotel, restaurant, accounting, masters, reports, and settings.
Hotel backend	backend/controllers/hotelController.js, backend/controllers/hotelController2CheckOut.js	Hotel masters, booking lifecycle, checkout, sales conversion, reports, room state, email, cancellation, and cross-module food posting.
Restaurant backend	backend/controllers/restaurantController.js	Item master, tables, KOT, direct sale, KOT payment conversion, print data, restaurant reports, and bill transfer.
Models	bookingModal.js, roomModal.js, kotModal.js, TableModel.js	Persistent records for bookings/check-ins/check-outs, rooms, KOTs, and tables.
Helpers	hotelHelper.js, restaurantHelper.js, checkoutHelper.js, saleCalculationHelper.js, nightAuditHelper.js	Filtering, room status, tax recalculation, payment receipts, settlements, mail, exports, and audit date validation.

Hotel Module

The hotel module starts with masters and configuration, then moves through booking, check-in, in-stay changes, checkout calculation, sale conversion, receipt/settlement posting, and reporting. The central records are Booking, CheckIn, and CheckOut, all backed by one extended schema. Rooms are held separately and status changes are synchronized through helper logic.

Hotel Operating Flow

Step	User / Screen	Backend Action	Data Impact
1	Masters and setup	Save additional pax, visit purpose, ID proof, food plan, rooms, room price levels, room type, floor, HSN, and hotel/restaurant config.	Creates master data used by booking and checkout calculations.
2	Room registration/list	addRoom, getRooms, getAllRooms, editRoom, deleteRoom.	Room records include type, bed, floor, HSN, unit, price levels, and status values: vacant, occupied, booked, dirty, blocked, household.
3	Booking	roomBooking generates hotel voucher numbers, validates availability, stores customer/guest, dates, rooms, food plan, additional pax, advances, and totals.	Booking document created; selected rooms and advance tracking establish the future stay.
4	Check-in	Check-in is handled by posting selected booking/check-in data into the shared booking schema with status changes and room occupancy.	Room becomes occupied/booked as applicable; guest identity and room details are retained.
5	During stay	swapRoom, updateBooking, updateCheckout, controlTaggedCheckIn, hold/release checkout, fetch current room status.	Room swap history, partial checkout history, hold arrays, and room status are updated.
6	Checkout preparation	checkoutWithArrayOfData and fetchOutStandingAndFoodData gather selected stays, advances, food bills, outstanding data, other charges, and payment allocations.	Checkout preview combines room, food plan, extra pax, restaurant, advance, discount, and payment data.
7	Checkout conversion	convertCheckOutToSale recalculates room tax, creates CheckOut, hotel sales voucher, TallyData, receipts, settlements, and updates rooms.	Rooms move toward dirty/available, sales and outstanding ledgers are posted, receipts settle paid amounts.
8	Post-checkout	Print/email APIs fetch specific data for checkout, sale, KOT, booking, or check-in; reports summarize sales, occupancy, cancellations, and login data.	Operational and accounting output becomes available for front office and management.

Key hotel calculation behavior

Checkout recalculation uses stay days per room, including room swap dates, then calculates taxable room amount, food plan totals, food plan tax, additional pax amount/tax, room total, grand total, and balance after advance. This matters because checkout does not simply reuse the original booking total; it recalculates the stay from selected room data before creating sales and tally entries.

Night-audit protection

The night-audit helpers normalize business dates, check whether an audited date locks a record, prevent tariff updates on locked dates, and restrict audit reopening to admin secondary users. This is an important control for preventing back-dated changes after daily closure.

Restaurant Module

The restaurant module is connected to both independent restaurant billing and hotel room-service billing. It uses product items as restaurant items, price levels to distinguish dine-in, takeaway, room service, and delivery pricing, KOT records for kitchen workflow, and sales/receipt/tally records for accounting output.

Restaurant Operating Flow

Step	User / Screen	Backend Action	Data Impact
1	Item master	addItem, updateItem, getItems, getAllItems, searchItems, exportItemsToExcel.	Creates product records with item code, image, category/subcategory, HSN/tax, unit, price levels, and default godown.
2	Table master	saveTableNumber, getTables, updateTable, deleteTable, updateTableStatus.	Tracks table number, description, and status. Occupied tables become effectively available after 24 hours through a model virtual.
3	KOT creation	generateKot creates a memoRandom voucher number, stores items/type/customer/food plan/kitchen batches, and occupies dine-in table.	KOT document is created as pending or completed depending on kotAutoApproval config.
4	KOT edit/print	editKot updates items, table, status, kitchen batches; frontend helpers generate kitchen tickets and customer bills.	Kitchen batches preserve printed item groups; table status follows dine-in movement.
5	Payment/direct sale	directSale and updateKotPayment convert KOTs or direct restaurant orders into sales, receipt/tally/settlement records, or room-posted credit.	Food sales are posted under restaurant voucher series and payment splits are retained.
6	Room service	KOT can store roomId and checkInNumber; getRestaurantBillsDetails maps KOTs to sale numbers for a checked-in guest.	Food bills can follow the guest to checkout and be included or settled there.
7	Transfer bills	transferKotBills moves selected KOT/sales records to another target room/check-in and updates party information.	Misposted restaurant bills can be reassigned before final hotel settlement.
8	Reports	getKotRegister, getSalesRegister, category-wise sales, date-wise item report.	Management can review KOTs, restaurant bill type, room number, food plan, payment type, customer, tax, discount, and cancellation state.

Restaurant payment behavior

The restaurant helper normalizes source types such as cash, bank, UPI, card, cheque, and credit. It recalculates KOT item tax/discount details and builds receipts for non-credit food split rows. When restaurant bills are part of a hotel checkout, the checkout helper can distribute food payments across restaurant sales and update their TallyData pending amount.

Shared Hotel and Restaurant Accounting Flow

The strongest integration point is checkout. Hotel checkout can include room charges, food plans, additional pax, other charges, discounts, restaurant bills, advances, and split payments. The newer checkout controller recalculates checkout data, separates room and food payment splits, creates sales vouchers, creates TallyData rows, creates receipts for non-credit payments, and creates Settlement records.

Flow Area	What Happens	System Impact
Voucher series	Hotel and restaurant use voucher series with `under` values such as hotel and restaurant for sales and receipts.	Numbers remain separated by operational area while sharing the accounting model.
Sales conversion	Hotel checkout creates salesModel rows from room data; restaurant direct/KOT payment creates restaurant sales rows from KOT items.	Both modules become standard sales vouchers for reports and accounting.
TallyData	Checkout and restaurant sales create bill rows with bill amount and pending amount.	Outstanding and receipt allocation logic can work consistently.
Receipts	Non-credit cash/bank/UPI/card/cheque splits create receiptModel records against billData.	Payments reduce pending amounts and provide print/report records.
Settlements	Receipt creation is accompanied by settlementModel rows pointing to cash/bank source parties.	Collection breakdown dashboards and source balances can be derived.
Credit / post-to-room	Credit splits do not create receipts immediately; restaurant food can be posted to room and settled at hotel checkout.	Outstanding remains visible until checkout or later receipt.

End-to-end checkout sequence

No.	Processing Step
1	Receive selected checkout rows, payment details, checkout mode, room assignments, restaurant base sale data, and discount/advance adjustments.
2	Recalculate each checkout row with stay days, room tax, food plan, additional pax, room total, grand total, and balance.
3	Fetch hotel sales voucher series and pre-fetch matching booking/check-in documents.
4	For each checkout item, determine customer party, payment mode, paid amount, pending amount, and room/food split arrays.
5	Create a CheckOut document and hotel Sales voucher with selected rooms as sales items.
6	Create TallyData rows; for split mode, non-credit room payments can become zero-pending rows while credit remains pending.
7	Update booking/check-in receipt references from booking/check-in numbers to sale numbers.
8	Create hotel and restaurant receipts/settlements for non-credit payments and update TallyData pending balances.
9	Mark full or partial checkout history, update remaining rooms, and sync room status through hotel helper logic.

Reports and Dashboards

The codebase includes operational dashboards and export/report APIs for daily management, finance, occupancy, and audit needs. Several frontend routes expose these as protected secondary-user screens.

Report / View	Data Covered
Hotel Dashboard and Summary Dashboard	Consolidated totals, revenue breakdown, daily/monthly collection, property sales, room count summary.
Hotel Flash Report	Date-based room and revenue snapshot with room metrics and restaurant details.
Bill Summary / View Report	Checkout or bill-level view by selected period and criteria.
Receipt Report	Receipt records by voucher series.
Login Report	Check-in records, status summary, created-by user, room swap and partial checkout history; exportable as PDF or Excel.
Tourist Report	Guest stay data including identity and travel/foreign-national fields.
Food Plan Report	Food plan usage and related totals.
Occupancy Checkout Report	Checked-out occupancy and revenue data.
Travel Agent / FO Sales Summary	Agent-related and front-office sales reporting.
Cancellation Report	Cancelled booking, check-in, checkout, KOT, receipt, and sale records in one report.
Restaurant KOT Register	KOT-level operational history and cancellation/payment state.
Restaurant Sales Register	Restaurant sales by bill type, room, food plan, payment type, item, tax, discount, customer, sponsor, and cancellation.

Controls and Strengths

Control	Observed Implementation
Authentication and company scope	Most APIs are protected by authSecondary and companyAuthentication checks, with secondary-user company access validation in night audit helpers.
Transactions	Critical writes use Mongoose sessions and transactions, especially room creation, KOT creation, KOT edit, and checkout conversion.
Night audit	Completed audit dates lock edits for hotel records/tariffs; reopening requires admin access and reason capture.
Voucher separation	Sales and receipts use voucher series, with hotel and restaurant series distinguished by `under` value.
Room/table status	Rooms track hotel lifecycle statuses; tables track restaurant availability and occupancy.
Cross-module correction	Room swap, partial checkout, bill transfer, hold/release, cancellation, and checkout edit APIs exist for real operational exceptions.

Recommended Improvements

The following recommendations are based on code structure and flow complexity, not on runtime testing against production data.

Priority	Recommendation	Reason
High	Split the very large hotel and restaurant controllers into service files by lifecycle: masters, booking, check-in, checkout, reports, restaurant billing, KOT/table.	The current controllers carry many responsibilities, which makes regression risk higher and slows review.
High	Standardize naming and folder spelling: `Restuarant` appears throughout frontend paths.	Consistent spelling lowers route/import mistakes and helps onboarding.
High	Add integration tests for checkout conversion with single, split, credit, room-service, partial checkout, and restaurant-posted bills.	Checkout is the highest-risk flow because it touches CheckOut, Sales, TallyData, Receipts, Settlements, rooms, bookings, and restaurant sales.
Medium	Centralize payment split normalization across hotel and restaurant.	Several functions normalize cash/bank/UPI/card/credit differently; a shared helper would reduce accounting edge cases.
Medium	Add explicit status enums for KOT and table status similar to ROOM_STATUS_VALUES.	This would prevent silent drift from typos or inconsistent casing.
Medium	Document voucher series requirements for hotel and restaurant setup.	Missing series causes hard failures at operational moments such as KOT, checkout, or receipt creation.
Low	Replace noisy console logging in controllers with structured logging guarded by environment.	Current logs may expose customer/payment data and make production debugging noisy.

Source Files Reviewed

backend/routes/secondaryUserRouters.js; backend/routes/kotRoutes.js; backend/controllers/hotelController.js; backend/controllers/hotelController2CheckOut.js; backend/controllers/restaurantController.js; backend/controllers/nightAuditController.js; backend/helpers/hotelHelper.js; backend/helpers/restaurantHelper.js; backend/helpers/checkoutHelper.js; backend/helpers/saleCalculationHelper.js; backend/helpers/nightAuditHelper.js; backend/models/bookingModal.js; backend/models/roomModal.js; backend/models/kotModal.js; backend/models/TableModel.js; frontend/src/routes/Routers.jsx; frontend/src/pages/Hotel; frontend/src/pages/Restuarant.

Conclusion

Camet ERP's hotel and restaurant modules are designed as an integrated hospitality workflow rather than two isolated modules. The hotel side manages the guest stay lifecycle and final accounting. The restaurant side manages food ordering, kitchen operations, table/room-service context, and food billing. The checkout layer joins them into one financial settlement process, using sales, TallyData, receipts, and settlements to make room, food, advance, discount, and split-payment activity reportable.